

Методическая разработка: Linux-router, nftables, NAT, port forwarding и WireGuard

1. Паспорт работы

Тема лекции: Linux-router, nftables, NAT, port forwarding и WireGuard VPN.

Учебный сценарий: настроить Linux-сервер как маршрутизатор между внешней и внутренней сетью, включить IP forwarding, применить базовую политику firewall через nftables, настроить NAT masquerade, port forwarding к внутреннему web-серверу и базовый WireGuard VPN-доступ.

ОС: Debian 12 Server или Ubuntu Server.

ПО: nftables, wireguard, iproute2, curl, tcpdump по возможности.

Формат: демонстрация преподавателя и самостоятельное повторение студентами.

Время: 4 академических часа.

Предварительная подготовка: есть web-сервер во внутренней сети, студенты понимают IPv4-адресацию, gateway, порт TCP/80 и базовый firewall.

Итоговый результат: rtr01 маршрутизирует трафик между сетями, внутренние клиенты выходят наружу через NAT, внешний клиент открывает внутренний web-сервер через port forwarding, WireGuard-клиент получает доступ к внутренней сети.

2. Что будет настроено

- Linux-router rtr01 с двумя интерфейсами;
- IP forwarding;
- базовые правила nftables;
- политика deny by default;
- NAT masquerade для LAN;
- port forwarding с внешнего порта 8080 на web01:80;
- WireGuard-интерфейс wg0;
- проверка маршрутизации, NAT, port forwarding и VPN.

3. Схема учебного стенда

```
internet-client
192.168.100.100
    |
    | external net 192.168.100.0/24
    v
enp0s3: 192.168.100.2
rtr01 Linux-router
enp0s8: 192.168.10.1
    |
    | internal LAN 192.168.10.0/24
    v
web01
192.168.10.20
```

WireGuard:

```
vpn-client 10.10.10.2/24 <--- UDP/51820 ---> rtr01 wg0 10.10.10.1/24
    |
    v
    LAN 192.168.10.0/24
```

Узел	Роль	Интерфейс/адрес
rtr01	Linux-router	enp0s3: 192.168.100.2/24, enp0s8: 192.168.10.1/24
web01	Внутренний web-сервер	192.168.10.20/24, gateway 192.168.10.1
internet-client	Внешний клиент	192.168.100.100/24
vpn-client	VPN-клиент	10.10.10.2/24

4. Исходные условия и правила безопасности

Перед началом:

- сделать снимок VM rtr01;
- проверить имена интерфейсов через `ip link`;
- не применять `deny by default` до разрешения SSH из сети управления;
- иметь консольный доступ к VM на случай ошибки firewall.

Проверка интерфейсов:

```
ip addr
```

Команда показывает интерфейсы и адреса. Нужно определить внешний и внутренний интерфейс.

В этой методичке:

- внешний интерфейс: `enp0s3`;
- внутренний интерфейс: `enp0s8`.

Если в вашей VM имена другие, нужно заменить их во всех командах.

5. Настройка адресов интерфейсов

Временная настройка для демонстрации:

```
sudo ip addr add 192.168.100.2/24 dev enp0s3
sudo ip addr add 192.168.10.1/24 dev enp0s8
sudo ip link set enp0s3 up
sudo ip link set enp0s8 up
```

Что делают команды:

- назначают адрес внешнему интерфейсу;
- назначают адрес внутреннему интерфейсу;
- включают интерфейсы.

Проверка:

```
ip addr show enp0s3
ip addr show enp0s8
```

На web01 gateway должен быть 192.168.10.1. Проверить:

```
ip route
```

Ожидаемый результат на web01:

```
default via 192.168.10.1
```

6. Включение IP forwarding

IP forwarding разрешает Linux пересылать пакеты между интерфейсами.

Включить временно:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Что делает команда:

- изменяет параметр ядра;
- разрешает маршрутизацию IPv4 до перезагрузки.

Проверить:

```
sysctl net.ipv4.ip_forward
```

Ожидаемый результат:

```
net.ipv4.ip_forward = 1
```

Сделать постоянно:

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-router.conf  
sudo sysctl --system
```

Что делают команды:

- записывают параметр в постоянный файл;
- применяют системные параметры.

7. Установка nftables

```
sudo apt update  
sudo apt install nftables -y  
sudo systemctl enable --now nftables
```

Что делают команды:

- устанавливают nftables;
- включают службу;
- запускают её сразу.

Проверка:

```
systemctl status nftables  
sudo nft list ruleset
```

Первая команда проверяет службу, вторая показывает текущие правила.

8. Базовый firewall deny by default

Создать конфигурацию:

```
sudo nano /etc/nftables.conf
```

Содержимое:

```
#!/usr/sbin/nft -f  
  
flush ruleset  
  
table inet filter {  
    chain input {  
        type filter hook input priority 0;  
        policy drop;  
    }  
}
```

```

    iif lo accept
    ct state established,related accept
    ip protocol icmp accept
    tcp dport 22 ip saddr 192.168.10.0/24 accept
    udp dport 51820 accept
}

chain forward {
    type filter hook forward priority 0;
    policy drop;

    ct state established,related accept
    ip saddr 192.168.10.0/24 oif enp0s3 accept
    ip daddr 192.168.10.20 tcp dport 80 accept
    ip saddr 10.10.10.0/24 ip daddr 192.168.10.0/24 accept
}

chain output {
    type filter hook output priority 0;
    policy accept;
}
}

```

Что важно:

- `policy drop` реализует `deny by default`;
- `ct state established,related accept` разрешает ответы на уже установленные соединения;
- SSH разрешён только из LAN;
- WireGuard разрешён на UDP/51820;
- `forward` разрешён из LAN наружу и к web-серверу.

Проверить синтаксис:

```
sudo nft -c -f /etc/nftables.conf
```

Если ошибок нет, применить:

```
sudo nft -f /etc/nftables.conf
```

Проверить:

```
sudo nft list ruleset
```

9. NAT masquerade

Добавить в `/etc/nftables.conf` после таблицы `inet filter`:

```

table ip nat {
    chain prerouting {
        type nat hook prerouting priority -100;
    }

    chain postrouting {
        type nat hook postrouting priority 100;
        oif enp0s3 ip saddr 192.168.10.0/24 masquerade
    }
}

```

Что означает правило:

- postrouting применяется перед выходом пакета наружу;
- oif enp0s3 выбирает внешний интерфейс;
- ip saddr 192.168.10.0/24 выбирает внутреннюю сеть;
- masquerade подменяет исходный адрес на адрес внешнего интерфейса router.

Проверить и применить:

```
sudo nft -c -f /etc/nftables.conf
sudo nft -f /etc/nftables.conf
```

Проверить выход из LAN с web01:

```
ping -c 4 192.168.100.100
```

Если есть интернет или внешний тестовый узел, проверить:

```
curl http://example.com
```

10. Port forwarding к внутреннему web-серверу

Задача: внешний клиент открывает `http://192.168.100.2:8080`, а router перенаправляет запрос на `web01:80`.

В таблицу `ip nat, chain prerouting`, добавить:

```
iif enp0s3 tcp dport 8080 dnat to 192.168.10.20:80
```

Итоговый фрагмент:

```
table ip nat {
    chain prerouting {
        type nat hook prerouting priority -100;
        iif enp0s3 tcp dport 8080 dnat to 192.168.10.20:80
    }

    chain postrouting {
        type nat hook postrouting priority 100;
        oif enp0s3 ip saddr 192.168.10.0/24 masquerade
    }
}
```

Проверить и применить:

```
sudo nft -c -f /etc/nftables.conf
sudo nft -f /etc/nftables.conf
```

На внешнем клиенте:

```
curl -I http://192.168.100.2:8080
```

Ожидаемый результат:

```
HTTP/1.1 200 OK
```

Если не работает, проверить:

```
sudo nft list ruleset
sudo ss -tulpen | grep ':8080'
```

Важно: `ss` может не показать порт 8080 как слушающий, потому что port forwarding выполняется firewall/NAT, а не отдельной службой.

11. Проверка маршрутизации и NAT

На rtr01:

```
ip route  
sudo nft list ruleset
```

На web01:

```
ip route  
ping -c 4 192.168.10.1  
curl -I http://192.168.100.2:8080
```

На внешнем клиенте:

```
ping -c 4 192.168.100.2  
curl -I http://192.168.100.2:8080
```

Если установлен tcpdump, на router можно посмотреть трафик:

```
sudo tcpdump -ni enp0s3 tcp port 8080
```

Команда показывает входящие подключения на внешний интерфейс.

12. Установка WireGuard

На rtr01:

```
sudo apt update  
sudo apt install wireguard -y
```

Создать ключи router:

```
wg genkey | tee rtr01_private.key | wg pubkey > rtr01_public.key  
chmod 600 rtr01_private.key
```

Что делают команды:

- `wg genkey` создаёт закрытый ключ;
- `wg pubkey` создаёт публичный ключ из закрытого;
- `chmod 600` ограничивает доступ к закрытому ключу.

Показать публичный ключ:

```
cat rtr01_public.key
```

13. Настройка WireGuard на router

Создать файл:

```
sudo nano /etc/wireguard/wg0.conf
```

Содержимое:

```
[Interface]  
Address = 10.10.10.1/24  
ListenPort = 51820  
PrivateKey = PASTE_RTR01_PRIVATE_KEY  
  
[Peer]  
PublicKey = PASTE_CLIENT_PUBLIC_KEY  
AllowedIPs = 10.10.10.2/32
```

Что означает:

- Address задаёт IP tunnel-интерфейса;
- ListenPort задаёт UDP-порт VPN;
- PrivateKey — закрытый ключ router;
- Peer описывает клиента;
- AllowedIPs указывает адрес клиента внутри tunnel.

Запустить:

```
sudo systemctl enable --now wg-quick@wg0
```

Проверить:

```
sudo wg show
ip addr show wg0
```

14. Настройка WireGuard-клиента

На VPN-клиенте установить WireGuard и создать ключи:

```
sudo apt install wireguard -y
wg genkey | tee client_private.key | wg pubkey > client_public.key
chmod 600 client_private.key
```

Создать конфигурацию клиента:

```
sudo nano /etc/wireguard/wg0.conf
```

Содержимое:

```
[Interface]
Address = 10.10.10.2/24
PrivateKey = PASTE_CLIENT_PRIVATE_KEY

[Peer]
PublicKey = PASTE_RTR01_PUBLIC_KEY
Endpoint = 192.168.100.2:51820
AllowedIPs = 10.10.10.0/24, 192.168.10.0/24
PersistentKeepalive = 25
```

Что означает:

- клиент получает tunnel-адрес 10.10.10.2;
- Endpoint указывает внешний адрес router;
- AllowedIPs направляет трафик к VPN-сети и LAN через tunnel;
- PersistentKeepalive полезен, если клиент находится за NAT.

Запустить:

```
sudo systemctl enable --now wg-quick@wg0
```

Проверить:

```
sudo wg show
ip route
```

15. Проверка VPN-доступа

С клиента:

```
ping -c 4 10.10.10.1
ping -c 4 192.168.10.20
curl -I http://192.168.10.20
```

Ожидаемый результат:

- ping до 10.10.10.1 проходит;
- ping до web01 проходит;
- curl получает HTTP/1.1 200 OK.

На router:

```
sudo wg show
```

Ожидаемый результат: виден peer и время последнего handshake.

16. Диагностика и типовые ошибки

Симптом	Возможная причина	Проверка	Исправление
LAN не выходит наружу	Forwarding выключен	sysctl net.ipv4.ip_forward	Включить forwarding
NAT не работает	Нет masquerade	nft list ruleset	Добавить postrouting masquerade
Port forwarding не работает	Нет DNAT или forward-разрешения	nft list ruleset	Проверить prerouting и forward
Потерян SSH	Deny by default без allow SSH	Консоль BM	Добавить SSH allow
WireGuard нет handshake	Неверные ключи или endpoint	wg show	Проверить public/private keys
VPN есть, LAN недоступна	Нет маршрута или forward-правила	ip route, nft list ruleset	Исправить AllowedIPs и forward

Журнал WireGuard:

```
journalctl -u wg-quick@wg0 -n 50 --no-pager
```

Журнал nftables:

```
systemctl status nftables
```

17. Итоговая самостоятельная проверка

- ☐ На rtr01 два интерфейса с правильными адресами.
- ☐ IP forwarding включён.
- ☐ nftables применяет deny by default.
- ☐ SSH разрешён из учебной сети.
- ☐ NAT masquerade работает.
- ☐ Port forwarding 192.168.100.2:8080 -> 192.168.10.20:80 работает.
- ☐ WireGuard wg0 запущен.
- ☐ VPN-клиент имеет handshake.
- ☐ VPN-клиент открывает внутренний web-сервер.

Финальные команды:

```
ip addr
ip route
sysctl net.ipv4.ip_forward
```



```
sudo nft list ruleset
curl -I http://192.168.100.2:8080
sudo wg show
curl -I http://192.168.10.20
```

18. Отчёт по работе

В отчёт включить:

1. Схему стенда.
2. Таблицу интерфейсов router.
3. Вывод `sysctl net.ipv4.ip_forward`.
4. Конфигурацию `nftables`.
5. Проверку NAT.
6. Проверку port forwarding.
7. Конфигурацию WireGuard без закрытых ключей.
8. Вывод `wg show`.
9. Описание одной ошибки и исправления.

В отчёт нельзя вставлять закрытые ключи WireGuard.

19. Контрольные вопросы

1. Для чего нужен IP forwarding?
2. Почему router имеет минимум два интерфейса?
3. Что означает deny by default?
4. Чем masquerade отличается от port forwarding?
5. Почему DNAT требует forward-разрешения?
6. Что показывает `nft list ruleset`?
7. Что такое peer в WireGuard?
8. Какие данные нельзя публиковать в отчёте?

20. Итог

После выполнения методички студент должен понимать Linux-router как связку маршрутизации, firewall, NAT и VPN. Рабочий результат подтверждается не одной командой, а цепочкой: интерфейсы, маршруты, forwarding, nftables, NAT, port forwarding, WireGuard handshake и доступ к внутреннему сервису.